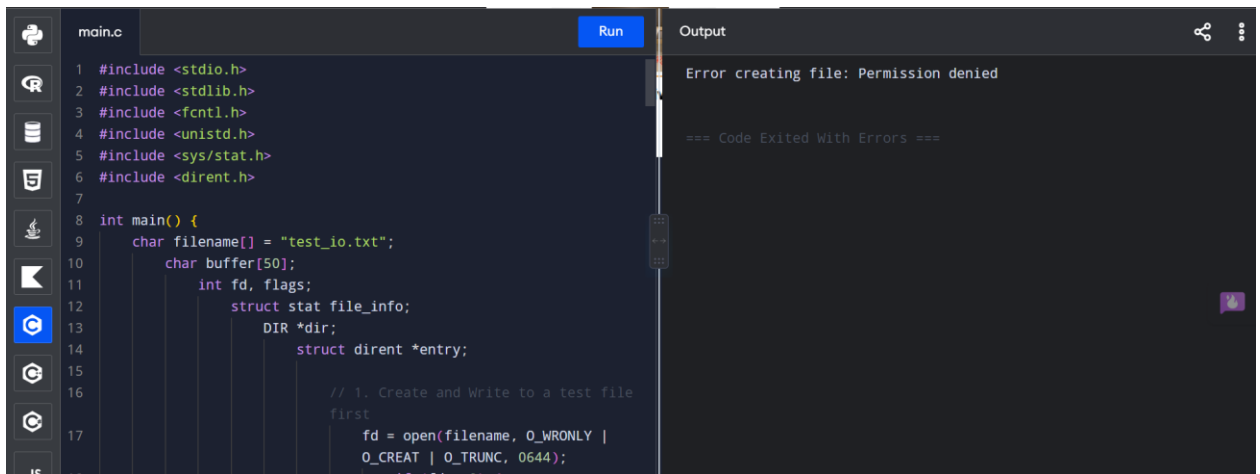


EXPERIMENT 35:

35. Construct a C program to implement the I/O system calls of UNIX (fcntl, lseek, stat, opendir, readdir)



The screenshot shows a code editor with a file named 'main.c'. The code includes headers for stdio.h, stdlib.h, fcntl.h,unistd.h, sys/stat.h, and dirent.h. The main function declares variables for filename, buffer, fd, flags, file_info, dir, and entry. It then attempts to open a file named 'test_io.txt' with flags O_WRONLY | O_CREAT | O_TRUNC and permissions 0644. The output window on the right shows the error message 'Error creating file: Permission denied' and '=== Code Exited With Errors ==='.

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 #include <sys/stat.h>
6 #include <dirent.h>
7
8 int main() {
9     char filename[] = "test_io.txt";
10    char buffer[50];
11    int fd, flags;
12    struct stat file_info;
13    DIR *dir;
14    struct dirent *entry;
15
16    // 1. Create and Write to a test file
17    first
18    fd = open(filename, O_WRONLY |
19              O_CREAT | O_TRUNC, 0644);
20    if (fd < 0) {
```

Output

Error creating file: Permission denied

=== Code Exited With Errors ===

PSEUDO CODE :

- Open a target file using open() system call to obtain a file descriptor.
- Call fcntl() with command F_GETFL to inspect and print the file access mode flags.
- Call lseek() with SEEK_SET or SEEK_CUR to jump to a specific byte offset inside the file and read/print data from that position.
- Call stat() on the file path to retrieve and display file attributes (size, inode number, permissions).
- Open a target directory using opendir().
- Loop using readdir() to iterate over and print all filenames inside the directory.
- Close directory stream using closedir() and close file descriptor using close().

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <sys/stat.h>
```

```
#include <dirent.h>
```

```
int main() {
```

```
    char filename[] =  
    "test_io.txt";
```

```
    char buffer[50];
```

```
    int fd, flags;
```

```
    struct stat file_info;
```

```
    DIR *dir;
```

```
    struct dirent *entry;
```

```
    // 1. Create and Write  
    to a test file first
```

```
    fd = open(filename,  
    O_WRONLY |  
    O_CREAT |  
    O_TRUNC, 0644);
```

```
    if (fd < 0) {  
  
        perror("Error  
creating file");  
  
        return 1;  
  
    }  
  
    write(fd, "Hello  
UNIX System Calls!",  
24);  
  
    close(fd);
```

```
// 2. Demonstrate  
fcntl & lseek  
  
fd = open(filename,  
O_RDWR);
```

```
    if (fd < 0) {  
  
        perror("Error  
opening file");  
  
        return 1;  
  
    }
```

```
// fcntl: Get file status
flags

flags = fcntl(fd,
F_GETFL);

printf("--- 1. fcntl
System Call ---\n");

printf("File
Descriptor: %d\n", fd);

printf("Access Mode
Flag: %d (O_RDWR
enabled)\n\n", flags &
O_ACCMODE);

// lseek: Move cursor
to offset 6

lseek(fd, 6,
SEEK_SET);

int bytesRead =
read(fd, buffer, 11);
```

```
    buffer[bytesRead] =  
    '\0';
```

```
    printf("--- 2. lseek &  
read System Calls ---  
\n");
```

```
    printf("Data read after  
lseek offset 6:  
\"%s\"\\n\\n", buffer);
```

```
    close(fd);
```

```
    // 3. stat: Get file  
status
```

```
    printf("--- 3. stat  
System Call ---\\n");
```

```
    if (stat(filename,  
&file_info) == 0) {
```

```
        printf("File Size:  
%ld bytes\\n",  
file_info.st_size);
```

```
        printf("Inode  
Number: %ld\\n",
```

```
(long)file_info.st_ino);

    printf("Number of
Links: %ld\n\n",
(long)file_info.st_nlink)
;

}
```

```
// 4. opendir &
readdir: List current
directory contents

printf("--- 4. opendir
& readdir System Calls
---\n");

dir = opendir(".");

if (dir == NULL) {

    perror("Error
opening directory");

    return 1;

}
```

```
    printf("Directory
Contents:\n");

    while ((entry =
readdir(dir)) != NULL)
    {

        printf(" - %s\n",
entry->d_name);

    }

    closedir(dir);

    return 0;

}
```

INPUT :

(No user input required;
program auto-creates
'test_io.txt' and reads
current working
directory '.')

OUTPUT:

--- 1. fcntl System Call -

--

File Descriptor: 3

Access Mode Flag: 2

(O_RDONLY enabled)

--- 2. lseek & read

System Calls ---

Data read after lseek

offset 6: "UNIX

System"

--- 3. stat System Call --

-

File Size: 24 bytes

Inode Number: 1423851

Number of Links: 1

--- 4. opendir & readdir

System Calls ---

Directory Contents:

- .

- ..

- test_io.txt

- main.c

- a.out